Marwadi University

BCA Semester – 3

05BC3301 –Database Management System-2 (DBMS-2)

Unit – 1: Introduction to PL/SQL

Topics

1. Overview
2. Advantages of PL/SQL
3. Generic PL/SQL Block
4. PL/SQL Environment
5. PL/SQL Literals
6. Data Types
7. Variables
8. Display Messages
9. Comments
10. Create and Execute PL/SQL
11. Example of PL/SQL Block
12. Control Structures

Overview of SQL

- SQL is so much powerful in handling data and various database objects.
- But, there is lack some of basic functionalities provided by other programming language.
- **For example,** SQL does not provide basic **procedural capabilities** such as **conditional checking, branching and looping**.
- Oracle provides **PL/SQL (Procedural Language / Structured Query Language) to overcome disadvantages of SQL**.
- PL/SQL supports all the functionalities provided by SQL along with its own **procedural capabilities**.
- **Data definition statements** are not allowed because PL/SQL code is **compile time**. So, it cannot **refer to objects that do not yet exist.**

SQL - Structured Query Language

- SQL is widely popular because it offers the following advantages –
- Allows users to access data in the relational database management systems.
- Allows users to describe the data.
- Allows users to define the data in a database and manipulate that data.
- Allows to embed within other languages using SQL modules, libraries & pre-compilers.
- Allows users to create and drop databases and tables.
- Allows users to create view, stored procedure, functions in a database.
- Allows users to set permissions on tables, procedures and views.

SQL Commands

- The standard SQL commands to interact with relational databases are CREATE, SELECT, INSERT, UPDATE, DELETE and DROP.
- These commands can be classified into the following groups based on their nature −

Sr.No.	Command & Description
1	**CREATE** Creates a new table, a view of a table, or other object in the database.
2	**ALTER** Modifies an existing database object, such as a table.
3	**DROP** Deletes an entire table, a view of a table or other objects in the database.

SQL Commands - Example

- **CREATE COMMAND: -**

This command is used to create database objects such as tables, views, indexes etc. Generally it is used to create the database tables. The syntax is as follows:

- **Syntax: -**

```
CREATE TABLE  <TABLE_NAME> (COLUMN_1 DATATYPE (SIZE),
COLUMN_2 DATATYPE (SIZE), COLUMN_N DATATYPE (SIZE));
```

- **Example: -**

```
CREATE TABLE EMP (NO NUMBER (3), NAME VARCHAR,
B_DATE DATE, SALARY NUMBER (6,2));
```

SQL Commands - Example

- **ALTER COMMAND: -**

The ALTER statement allows the user to make some kind of change to structure of some object in the database. The ALTER statement syntax can then be simplified to one of two following statements:

- **Syntax: -**

```
ALTER TABLE <TABLE_NAME>
ADD (NEWCOLUMN_NAME DATATYPE (SIZE), NEWCOLUMN_NAME DATATYPE
(SIZE));
                    OR
ALTER TABLE<TABLE_NAME>
MODIFY (COLUMN_NAME DATATYPE (NEW_SIZE));
```

- **Example: -**

```
ALTER TABLE EMP ADD
(GENDER CHAR (1), DEPT VARCHAR2 (15));

ALTER TABLE EMP MODIFY (NO NUMBER (5));
```

SQL Commands - Example

- **DROP COMMAND: -**

  When a table is no longer needed, it can be dropped. Both the table definition and the rows in the table are dropped, and the space allocated for the table is made available for other database objects. The syntax for the DROP statement is about as simple as it gets:

  - **Syntax: -**

    DROP TABLE <TABLE_NAME>;

  - **Example: -**

    DROP TABLE EMP;

SQL Commands

- DML - Data Manipulation Language

Sr.No.	Command & Description
1	**SELECT** Retrieves certain records from one or more tables.
2	**INSERT** Creates a record.
3	**UPDATE** Modifies records.
4	**DELETE** Deletes records.

SQL Commands - Example

- **SELECT COMMAND: -**

In its most basic form, the SELECT statement has a list of columns to select from a table, using the SELECT ... FROM syntax. The * means "all columns." The most basic SELECT syntax can be described as follows:

- **Syntax: -**

SELECT [DISTINCT] COLUMN_1,COLUMN_2 , …..,COLUMN_N
FROM <TABLE_NAME>
[WHERE <Condition>] [Order by <Column> [Asc / Desc}];

NOTE: - It displays the information contained in the column, which is selected in the query only.

OR

SELECT * FROM <TABLE_NAME>;

NOTE: - It displays the information contained in all the fields of the table.

SQL Commands - Example

SELECT * FROM <TABLE_NAME> WHERE <CONDITION>;

NOTE: - It displays the information, which matches with the condition given in the query.

- **Example: -**

```
SELECT NO, NAME, GENDER FROM EMP;
SELECT * FROM EMP;
SELECT * FROM EMP WHERE NAME = 'Ram';
SELECT * FROM EMP WHERE SALARY=8000;
```

- **Other Examples: -**

```
SELECT NAME, NO FROM EMP;
SELECT DISTINCT NO FROM EMP;
SELECT DISTINCT NAME FROM EMP;
SELECT DISTINCT NO, NAME FROM EMP;
SELECT * FROM EMP WHERE NO>200;
SELECT * FROM EMP WHERE NO=200;
SELECT * FROM EMP ORDER BY NAME;
SELECT DISTINCT NAME FROM EMP ORDER BY NAME;
SELECT DISTINCT NO, NAME FROM EMP ORDER BY NO;
SELECT DISTINCT NO, NAME FROM EMP ORDER BY NAME;
SELECT * FROM EMP ORDER BY NAME DESC;
SELECT * FROM EMP ORDER BY NO, NAME;
SELECT * FROM EMP ORDER BY NO, NAME DESC;
SELECT * FROM EMP ORDER BY NO DESC, NAME;
SELECT * FROM EMP ORDER BY NO DESC, NAME DESC;
```

SQL Commands - Example

- **INSERT COMMAND: -**

  This is used to add one or more row's (record's) into the table. While using this command values are separated by commas.

  - **Syntax: -**

    INSERT INTO <TABLE_NAME> VALUES (V1, V2, ……);

    OR

    INSERT INTO <TABLE_NAME> VALUES
    (&COLUMN_NAME, '&COLUMN_NAME', ……) ;

  - **Example: -**

    INSERT INTO EMP VALUES (100,'Ram','16-MAR-06', 10000,'M');

    INSERT INTO EMP (NO, NAME) VALUES (200,'Sita');

    OR

    INSERT INTO EMP VALUES
    (&NO,'&NAME','&DATE', &SALARY,'&GENDER');

    NOTE: - Write character (including varchar2) and Date fields inside ' '. For e.g. '&NAME','&DATE'.

SQL Commands - Example

- **UPDATE COMMAND: -**

  An UPDATE statement will change one or more rows in a database table. The basic form of an UPDATE statement must specify which table to update, which column(s) to change, and optionally, whether to change all the rows in the table or just a few. The syntax is as follows:

  - **Syntax: -**

    UPDATE <TABLE_NAME> SET <COLUMN_NAME>=VALUE;

    OR

    UPDATE <TABLE_NAME>
    SET <COLUMN_NAME1>=VALUE1,<COLUMN_NAME2>=VALUE2
    WHERE COLUMN_NAME=VALUE;

  - **Example: -**

    UPDATE EMP SET SALARY = 8000;

    NOTE: - The above command will update the Salary's of all the employees to Rs.8000.

    UPDATE EMP SET NO=300 WHERE NAME= 'Ravi';
    UPDATE EMP SET NO=500, B_DATE='3-June-04' WHERE NAME='Ram';

    NOTE: - The above command will update only those rows, which fall under the condition, and not all the rows of the table.

SQL Commands - Example

- **DELETE COMMAND: -**

  As the name implies, the DELETE statement will remove rows from a database table. You can delete all rows or use a WHERE clause to specify rows, similar to the UPDATE statement. Here's the syntax:

  - **Syntax: -**

    DELETE FROM <TABLE_NAME>;
    DELETE FROM <TABLE_NAME> WHERE COLUMN_NAME=VALUE;

  - **Example: -**

    DELETE FROM EMP;
    DELETE FROM EMP WHERE NO=100;

SQL Commands

- DCL - Data Control Language

Sr.No.	Command & Description
1	**GRANT** Gives a privilege to user.
2	**REVOKE** Takes back privileges granted from user.

SQL Commands - Example

- **GRANT COMMAND: -**

The GRANT statement is almost self-explanatory. GRANT will give a privilege (either object or system) to a user, a role, or to all users. The basic syntax for granting privileges is as follows:

- **Syntax: -**

```
GRANT sys_privilege [, sys_privilege...]
TO user | role | PUBLIC [, user | role | PUBLIC...];

GRANT obj_privilege [(column list)] ON object
TO user | role | PUBLIC [WITH GRANT OPTION];
```

- **Example: -**

```
GRANT CONNECT TO BCA;
GRANT RESORUCE TO BCA;
GRANT DBA TO MCA;
GRANT SELECT ON EMP TO MYROLE;
GRANT MYROLE TO MCA;
```

SQL Commands - Example

- **REVOKE COMMAND: -**
- As you would expect, the REVOKE statement is the opposite of the
- GRANT statement. Either system privileges or object privileges can be
- revoked with the following basic syntax:

- **Syntax: -**
- REVOKE obj_privilege | ALL [, obj_privilege] ON object
- FROM user | role | PUBLIC [, user | role | PUBLIC...];
- REVOKE sys_privilege | ALL [, sys_privilege...]
- FROM user | role | PUBLIC [, user | role | PUBLIC...];

- **Example: -**
- REVOKE CONNECT FROM BCA;
- REVOKE REOSOURCE FROM BCA;
- REVOKE DBA FROM MCA;

SQL Commands

- TCL – Transaction Control Language
- Transaction is a sequence of SQL statements that Oracle treats as a single block of unit. To save a set of changes made to a table using DML commands and to get back to the original state before changes have been made we use TCL.

- Commands available in DCL are as follows:
- **1. COMMIT**
- **2. ROLLBACK**
- **3. SAVEPOINT**

SQL Commands - Example

- **COMMIT COMMAND: -**
- A set of changes made to a table using DML (INSERT, UPDATE, DELETE) commands are temporary. To make the changes permanent, COMMIT command must be needed after DML commands.
- **Syntax: -**
- COMMIT;
- **Example: -**
- DELETE FROM EMP WHERE NO=500;

  COMMIT;

SQL Commands - Example

- **ROLLBACK COMMAND: -**
- The ROLLBACK statement allows you to change your mind about a transaction. It brings back the original state of the tables to the state as of the last COMMIT statement or the beginning of the current transaction.
- **Syntax: -**
- ROLLBACK [TO SAVEPOINT <Savepoint_Name>];
- **Example: -**
- DELETE FROM EMP WHERE NO=500;

  ROLLBACK;
- **NOTE: -** You cannot ROLLBACK the transaction after the COMMIT. So, After the COMMIT you cannot use ROLLBCAK command.

SQL Commands - Example

- **SAVEPOINT COMMAND: -**
- Additionally, you can use SAVEPOINT to further subdivide the DML statements within a transaction before the final COMMIT of all DML statements within the transaction. SAVEPOINT essentially allows partial rollbacks within a transaction.

- **Syntax: -**
- SAVEPOINT <Savepoint_Name>;
- ROLLBACK TO SAVEPOINT <Savepoint_Name>;
- **Example: -**
- DELETE FROM EMP WHERE NO=500;
- SAVEPOINT A1;
- ROLLBACK TO SAVEPOINT A1;

DBMS - II

Let's get started...

Advantages of PL/SQL

1) Procedural Capabilities
2) Support to variables
3) Error Handling
4) User Defined Functions
5) Portability
6) Sharing of Code
7) Efficient Execution

Advantages of PL/SQL

1) **Procedural Capabilities**

- PL/SQL provides procedural capabilities such as **condition checking, branching and looping.**
- This enables programmer to **control execution** of a program based on some conditions and user inputs.

2) **Support to variables**

- PL/SQL supports **declaration and use of variables**.
- These variables can be used to **store intermediate results** of a query or some expression.

3) **Error Handling**

- When an error occurs, **user friendly message can be displayed**.
- **Execution of program can be controlled** instead of abruptly terminating the program.

Advantages of PL/SQL

4) **User Defined Functions**

- SQL also supports **user defined functions and procedures**.

5) **Portability**

- Programs written in PL/SQL are portable.
- It means, programs can be **transferred and executed from any other computer hardware and operating system, where Oracle is operational.**

6) **Sharing of Code:**

- Allows user to **store compiled code** in database.
- PL/SQL code can be **accessed and shared by different applications.**
- PL/SQL code can be **executed by other programming language** like JAVA.

Advantages of PL/SQL

7) Efficient Execution

- PL/SQL sends an entire block of SQL statements to the Oracle engine, where these statements are executed in one go.
- Reduces **network traffic and improves efficiency of execution**.
- In **SQL, all statements are transferred one by one.**

Generic PL/SQL Block

- PL/SQL code is grouped into structures called **block**.
- A block is called a **named block**, if it is given particular name to identify.
- A block is called an **anonymous block**, if it is not given any name.
- **Named blocks** are created while creating database objects such as **function, procedure, package and trigger.**

```
DECLARE
        <Declaration Section>          } Optional
BEGIN
        <Executable Commands>          } Mandatory
EXCEPTION
        <Exception Handling>           } Optional
END ;                                  } Mandatory
```

Generic PL/SQL Block

1) **Declarations (Optional)**

- This section starts with the keyword '**DECLARE'.**
- It defines and initializes **variables and cursors used in the block.**

2) **Begin (Executable Commands) (Mandatory)**

- This section starts with the keyword '**BEGIN'.**
- It contains various SQL and PL/SQL statements providing functionalities like **data retrieval, manipulation, looping and branching.**

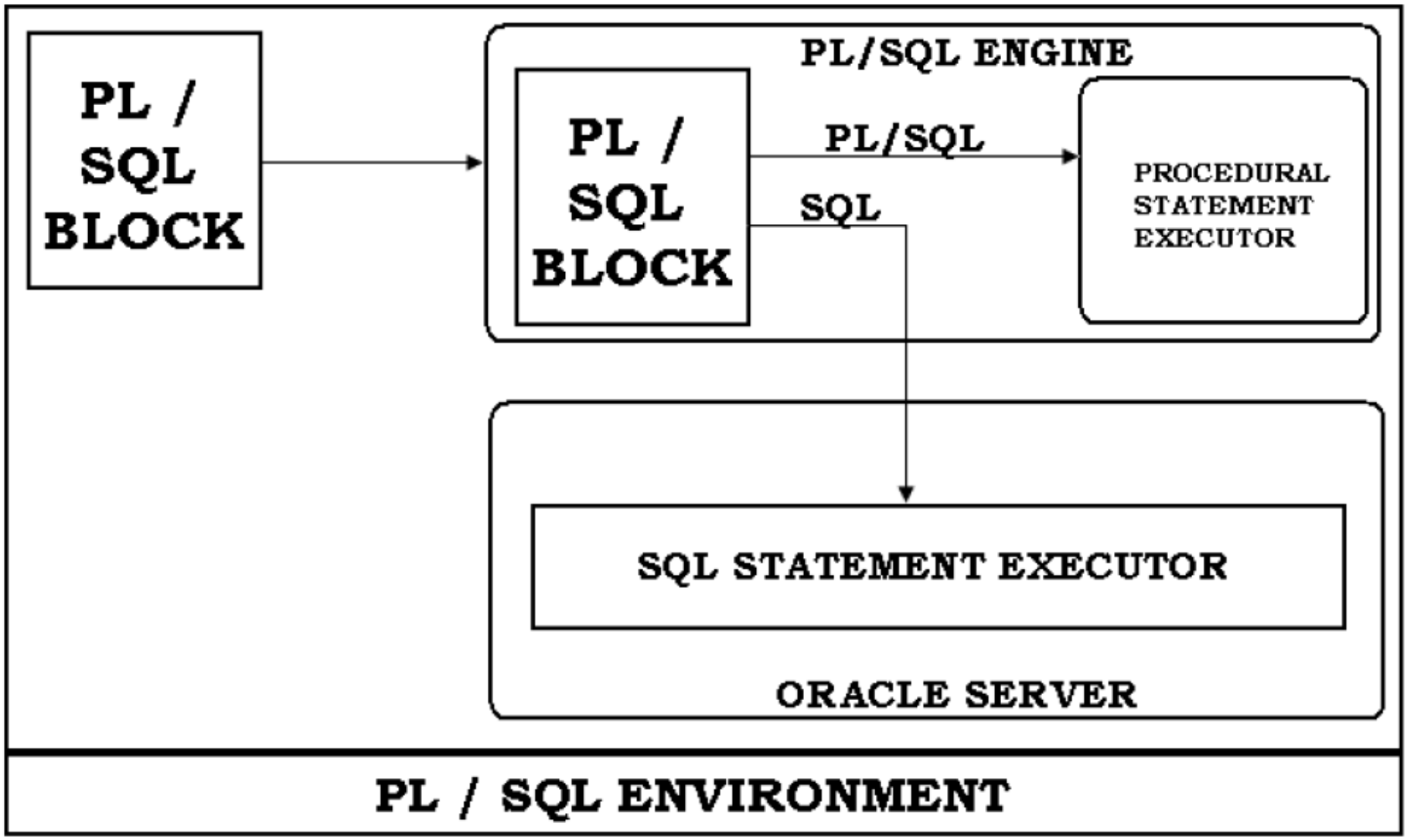
3) **Exception Handling (Optional)**

- This section starts with the keyword '**EXCEPTION'.**
- It **handles errors** that arise during the **execution of data manipulation statements in 'executable commands' section.**

4) **End (Mandatory)**

- This section specifies that **end** of PL / SQL block.

PL/SQL Environment



PL/SQL Environment

- **PL / SQL Execution Environment**
- The PL/SQL engine accepts as input any valid PL/SQL unit. The engine runs procedural statements, but sends SQL statements to the SQL engine in the database, as shown in the figure above.
- Typically, the database processes PL/SQL units. When an application development tool processes PL/SQL units, it passes them to its local PL/SQL engine.
- If a PL/SQL unit contains no SQL statements, the local engine processes the entire PL/SQL unit.
- This is useful if the application development tool can benefit from conditional and iterative control.

The PL/SQL Literals

- A literal is an explicit numeric, character, string, or Boolean value not represented by an identifier.
- For example, TRUE, 786, NULL, ‘marwadi university' are all literals of type Boolean, number, or string.
- PL/SQL, literals are case-sensitive. PL/SQL supports the following kinds of literals −
  - Numeric Literals
  - Character Literals
  - String Literals
  - BOOLEAN Literals
  - Date and Time Literals

The PL/SQL Literals

- **Numeric Literals:**
- 25 6.34 7g2 .1 +17 -5
- 050 78 -14 0 +32767
- 6.6667 0.0 -12.0 3.14159 +7800.00

- **Character Literals**
- 'A' '%' ‘*’ 'z' '('

The PL/SQL Literals

- **String Literals**
- 'Hello, world!'
- ‘Marwadi Univerisity'
- '19-NOV-12'

- **BOOLEAN Literals**
- TRUE, FALSE, and NULL.

- **Date and Time Literals**
- DATE ‘25-DEC-78';
- TIMESTAMP '2012-10-29 12:01:01';

Data Types

- PL/SQL is **super set** of the SQL.
- So, it supports all the data types provided by SQL.
- In PL/SQL Oracle provides **subtypes of the data types.**
- **For example,** the data type NUMBER has a subtype called INTEGER.

Category	Data Type	Sub types/values
Numerical	NUMBER	BINARY_INTEGER, DEC, DECIMAL, DOUBLE PRECISION, FLOAT, INTEGER, INT, NATURAL, POSITIVE, REAL, SMALLINT
Character	CHAR,LONG, VARCHAR2	CHARACTER, VARCHAR, STRING, NCHAR, NVARCHAR2
Date	DATE	-
Binary	RAW, LONG RAW	-
Boolean	BOOLEAN	Can have value like TRUE, FALSE and NULL
RowID	ROWID	Stores values of address of each record

Variables

- Variables are used to store values that may be change during the execution of program.
- In PL/SQL, variables contain values resulting from **queries or expression**.
- Variables are declared in **Declaration section** of the PL/SQL block.
- It can assign valid data type and can also be initialized if necessary.

Variables

Declare a Variable

Syntax:

variableName datatype [NOT NULL] := initialValue;

- **datatype** can be any valid data type supported by PL/SQL.
- **':= '** used as **assignment operator**.
- If variable need to be initialized then **initialValue** can be assigned at declaration time.
- If **NOT NULL** is included in declaration, variable cannot have NULL value during program execution and such **variable must be initialized**.

Variables

Example:

no **NUMBER(3);**

value **DECIMAL;**

city **CHAR(10);**

name **VARCHAR(10);**

counter **NUMBER(2) NOTNULL := 0**

Variables

Anchored Data Type

- A variable can be declared as **anchored data type.** It means, **datatype** for variable is **determined based on the data type of the other object.**
- This object can be other **variable or a column of the table.**
- This provides ability to match the **data types of the variables** with the **data types of the columns** defined in the database.
- If data type of column is changed, then the data type of variable will also **changed automatically**.

Advantage: It **reduces maintenance cost** and allow a program to adapt changes made in tables.

Variables

Anchored Data Type

Syntax:

variableName object%TYPE [NOTNULL] := initialValue ;

- **object** can be any variable declared previously or column of a database.
- To refer of a column of particular table, column name must be combined with table name.

Example

no **Account.Acc_No%TYPE := 101;**

bal **Account.Balance%TYPE;**

name **Customer.name%TYPE;**

Variables

Declare a Constant

- A constant is also used to store value like a **variable**.
- But, unlike variable, a value stored in constant **cannot be changed during program execution**.
- A constant **must be initialized** at declaration time.

Syntax:

constantName CONSTANT datatype := initialValue ;

Example

pi **CONSTANT NUMBER(3,2) := 3.14 ;**

Variables

Assign Value to Variable

1) **By using Assignment Operator**

   Syntax:

   variableName **:=** **value** ;

   - A value can be a **constant value or result of some expression or return value of some function**.

2) **By Reading from the Keyboard:**

   Syntax:

   variableName **:=** **&variableName** ;

   - This is similar to **scanf() function.**
   - Whenever **'&'** is encountered, a **value read from the keyboard** and assign it to variable.

   no := &no;

Variables

Assign Value to Variable

3) Selecting or Fetching table data values into Variables:

Syntax:

SELECT col1, col2, …, colN INTO var1, var2, …, varN FROM tableName **WHERE condition ;**

- This statement retrieves values for **specified column and stores** them in given variable.
- **Data type and size of variables** must be **compatible with the relative columns**.
- A condition in **WHERE** clause must be such that it selects **only single record**.
- This statement cannot work if **multiple records are selected**.

Variables

Assign Value to Variable

3) Selecting or Fetching table data values into Variables:

Example: Write PL/SQL block stores account number and balance for account '**A01**' into variable '**no**' and '**bal**'.

Input:

```
DECLARE
        no    Account.Acc_No%TYPE;
        bal   Account.Balance%TYPE;
BEGIN
        SELECT Acc_No, Balance INTO no, bal FROM Account
        WHERE  Acc_No = 'A01' ;
END;
/
```

Display Messages

Syntax:

```
dbms_output.put_line ( message );
```

- A **dbms_output** is a **package**, which provides functions to accumulate information in a buffer.
- A **put_line** is a **function**, which display messages on the screen.
- A **message** is a **character string to be displayed**.
- To display **data of other data type**, they must be **concatenated with some character string**.
- The environment parameter, **SERVEROUTPUT** must be **ON** to display messages on screen.

```
SET SERVEROUTPUT ON;
```

Display Messages

Example:

```
dbms_output.put_line ( 'Hi Hello World…' );
```

Hi Hello World…

Concatenation Operator

```
dbms_output.put_line ( 'Sum =' || 25 );
```

Sum = 25

```
dbms_output.put_line ( 'Square of ' || 3 || ' is ' || 9 );
```

Square of 3 is 9

Comments

- Comments are statement that will **not get executed** even though they are present in the program code.
- Comments are used to **increase readability** of a program.
- There are two types of comments:

1) -- (Double hyphen or double dash) (Single Line Comment):

- Treats single line as a comment.

```
-- This single line is a comment
```

2) /* …. */ (Multiple Line Comment):

- Treats multiple lines as comment.

```
/* This statement is spread over two line and both lines are treated as comments */
```

Create and Execute PL/SQL Block

- An **EDIT** command can be used on **SQL prompt** to **open a notepad** from the **SQL * PLUS environment**.
- The following syntax creates and opens a file:

  EDIT filename

Example:

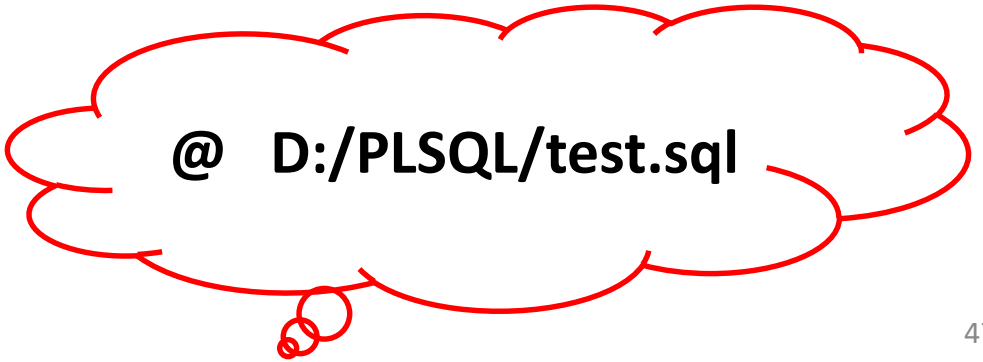
  EDIT D:/PLSQL/test.sql

- Create and open a file named **'test.sql'.**
- Write a program code or statements in a file and save it.
- File should have **'.sql'** extension and last statement in file should be **'/'.**
- To **execute** this block:

  RUN filename **OR**

  START fileName **OR**

  @ fileName



Example of PL/SQL Block

- **Write a program to calculate the AREA and store that value in the table AREAS (RADIUS NUMBER (5), AREA NUMBER (14,2)).**

```
SET SERVEROUTPUT ON
DECLARE
    PI CONSTANT NUMBER (9,7):=3.1415927;
    RADIUS NUMBER (5);
    AREA NUMBER (14,2);

BEGIN
    RADIUS:= 3;
    AREA:= PI * POWER (RADIUS, 2);

    INSERT INTO AREAS VALUES (RADIUS, AREA);
    DBMS_OUTPUT.PUT_LINE ('AREA IS: ' || AREA);
END;
/
SET SERVEROUTPUT OFF
```

Control Structures

- There are 3 types of Control Structures in PL/SQL:

1) **Conditional Control**
2) **Iterative Control**
3) **Sequential Control**

Control Structures

1) Conditional Control

- To control the **execution of block of code** based on some condition, PL/SQL provides the **IF statement.**
- The **IF – THEN – ELSIF – ELSE – END IF** construct can be used to execute specific part of the block based on the condition provided.

Syntax:

```
IF    condition    THEN
        -- Execute commands
ELSIF   condition    THEN
        -- Execute command . .
ELSE
        -- Execute command
END IF;
```

Control Structures

1) Conditional Control

Example: Write a program to read a number from user and determine whether it is odd or even.

Input:

```
DECLARE
        no    NUMBER;
BEGIN
        dbms_output.put_line ( 'Enter value for no: ' );
        no := &no;
        IF   MOD (no, 2) = 0   THEN
                dbms_output.put_line ( 'Given Number '|| no || ' is EVEN.' );
        ELSE
                dbms_output.put_line ( 'Given Number '|| no || ' is ODD.' );
        END IF ;
END ;
/
```

```
Enter value for no: 7
Old 5: no := &no;
New 5: no := 7;
Given Number 7 is ODD.
```

Control Structures

1) Conditional Control

Example: Write a program to debit a given account. Read account number and amount to be debited. Debit the balance if the resulting balance is not less than zero. This means, a balance in account should not go to negative while withdrawing amount .

Control Structures

2) Iterative Control

- Iterative control allows a group of statements to execute **repeatedly in a program.** It is called **Looping**.
- PL/SQL provides three constructs to implement loops, as listed below:

  1. LOOP

  2. WHILE

  3. FOR

- In PL/SQL, any loop starts with a **LOOP** keyword and it terminates with an **END LOOP** keyword.
- Each loop requires a **conditional statement** to control the **number of times a loop** is executed.

Control Structures

2) Iterative Control

1. LOOP

Syntax:

```
LOOP
        -- Execute commands..
END LOOP;
```

- **LOOP** is an infinite loop. It executes commands in its **body infinite times**.
- So, it requires an **EXIT** statement within its body to terminate the **loop after executing specific iteration**.

Control Structures

2) Iterative Control

1. LOOP

Example: Display number from 1 to 5 along with their square values using LOOP construct.

Output:

Value	Square
1	1
2	4
3	9
4	16
5	25

Input:

```
DECLARE
        counter   NUMBER(3)   :=   1 ;
BEGIN
        LOOP
        output dbms_output.put_line ('Value ' || ' Square');
        EXIT   WHEN   counter   >   5;
                dbms_output.put_line ( ''||counter || ' ' || counter*counter);
                counter   :=   counter + 1;
        END  LOOP;
END ;
/
```

Control Structures

2) Iterative Control

2. WHILE

Syntax:

```
WHILE   Condition
LOOP
        -- Execute Commands..
END LOOP;
```

- The **WHILE** loop executes commands in its body as long as the condition remains **TRUE**.
- The loop terminates when the **condition** evaluates to **FALSE or NULL**.
- The **EXIT** statement can also be used to exit the loop.

Control Structures

2) Iterative Control

2. WHILE

Example: Display numbers from 1 to 5 along with their square values using WHILE construct.

Input:

```
DECLARE
        counter   NUMBER(3)  :=  1;
BEGIN
        dbms_output.put_line (‘ Value ’ || ‘ Square’);
        WHILE  counter  <=  5
        LOOP
                dbms_output.put_line ( ‘ ’ ||counter|| ‘ ’ ||
                                        counter * counter);
                counter  :=  counter + 1;
        END LOOP;
END;
/
```

Output:

Value	Square
1	1
2	4
3	9
4	16
5	25

Control Structures

2) Iterative Control

3. FOR

Syntax:

```
FOR    variable   IN    [ REVERSE ]   start .. end
LOOP
        -- Execute command
END LOOP;
```

- Here, a **variable** is a **loop control variable**. It is declared **implicitly** by PL/SQL.
- The **FOR LOOP variable** is **always incremented by 1** and any other increment value cannot be specified.
- A **start and end** specifies the **lower and upper bound** for the loop control variable.
- If **REVERSE** keyword is provided, loop is executed in **reverse order.**

Control Structures

2) Iterative Control

3. FOR

Example: Display numbers from 1 to 5 along with their square values using FOR construct.

Input:

```
DECLARE
        counter   NUMBER(3) := 1;
BEGIN
        dbms_output.put_line (' Value ' || ' Square');
        FOR   counter  IN   1 .. 5
        LOOP
                dbms_output.put_line ( ' ' || counter || ' ' ||
                                        counter * counter);
        END LOOP;
END;
/
```

Output:

Value	Square
1	1
2	4
3	9
4	16
5	25

Control Structures

3) Sequential Control

- Normally, execution proceeds sequentially within the block of code.
- Sequence can be changed conditionally as well as **unconditionally**.
- To alter the sequence **unconditionally,** the **GOTO statement** can be used.

Syntax:

```
GOTO    jumpHere ;
          :
          :
<< jumpHere >>
```

- The **GOTO statement** makes flow of execution to jump at **<< jumpHere >>.**

Control Structures

3) Sequential Control

Example: The following code illustrates the use of the GOTO statement.

Input:

```
BEGIN
        dbms_output.put_line ('Code Starts.');
        dbms_output.put_line ('Before GOTO statement..');
        GOTO   jump;
                dbms_output.put_line ('This statement will not get
                                        executed..');
        << jump >>
        dbms_output.put_line ('Flow of execution jumped here..');
END;
/
```

Output:

Code Starts.
Before GOTO Statement..
Flow of execution jumped here..